

```

1
2 #include "Accelerometre.h"
3 #include "mcc_generated_files/spi1.h"
4 #include "mcc_generated_files/pin_manager.h"
5
6 #define DUMMY 0xA5
7
8 //Initialize the MC3419 sensor and set as the default configuration
9 void MC3419_start(void)
10 {
11     //Init Reset
12     MC3419_reset();
13     MC3419_stop();
14
15     //Range: 2g
16     MC3419_SetRangeCtrl(Range2g);
17     //Sampling Rate: 50Hz by default
18     MC3419_SetSampleRate(ODR100);
19     // Set polarity for INT1
20     MC3419_Pol_INT(GPIO1_INTN1_IAH | GPIO1_INTN1_IPP);
21     // All interrupt on INT1
22     //MC3419_FiFo_CTRL_REG(COMB_INT_EN);
23     // Set active interrupt
24     MC3419_INT_Enable(SHAKE_INT_EN);
25     // Set motion control
26     MC3419_MotionCTRL(SHAKE_EN | ANYM_EN);
27     // set pending
28     MC3419_ReadIntStatusRegisterAndAck();
29     // Set de Shake threshold
30     MC3419_WriteShakeThresholdRegister(1000);
31     // Set de Shake Duration and P2P
32     MC3419_WriteShakeDurationAndP2PRegister(1, 50);
33
34     //Mode: Active
35     MC3419_wake();
36     delay(50);
37 }
38
39 // Set the operation mode
40 void MC3419_SetMode (MC3419_Mode Mode)
41 {
42     uint8_t value;
43
44     value = ReadRegister8(addr_MODE);
45     value &= Mask_Mode;
46     value |= Mode;
47
48     writeRegister8(addr_MODE, value);
49 }
50
51 // Set the range control
52 void MC3419_SetRangeCtrl (MC3419_Range Range)
53 {
54     uint8_t value;
55     MC3419_SetMode(STANDBY);
56     value = ReadRegister8(addr_Range);
57     value &= Mask_Range;
58     value |= (Range << 4) & 0x70;
59     writeRegister8(addr_Range, value);
60 }
61
62 // Set the sampling rate
63 void MC3419_SetSampleRate(MC3419_SamplRate SampleRate)
64 {
65     uint8_t value;
66     MC3419_SetMode(STANDBY);
67     value = ReadRegister8(addr_SR);
68     value &= 0x00;
69     value |= SampleRate;
70     writeRegister8(addr_SR, value);
71 }
72 // Set mode wake
73 void MC3419_wake(void)

```

```

74 {
75     //Set mode as wake
76     MC3419_SetMode(WAKE);
77 }
78 // Set mode Standby
79 void MC3419_stop(void)
80 {
81     //Set mode as Sleep
82     MC3419_SetMode(STANDBY);
83 }
84 //Initial reset
85 void MC3419_reset(void)
86 {
87     // Stand by mode
88     MC3419_stop();
89
90     delay(10);
91
92     // power-on-reset
93     writeRegister8(addr_Reset, Reset);
94
95     delay(50);
96
97     // Disable interrupt
98     writeRegister8(addr_INTR_CTRL,
99                     INTR_Desable);
100 // delay(10);
101 // // 1.00x Analog Gain
102 // writeRegister8(0x2B, 0x00);
103 // delay(10);
104 // // DCM disable
105 // writeRegister8(0x15, 0x00);
106
107     delay(50);
108 }
109
110 // Read ID
111 uint8_t MC3419_ID (void)
112 {
113     return ReadRegister8(addr_Chip_ID);
114 }
115 // Read Register
116 uint8_t ReadRegister8(uint8_t address)
117 {
118     uint8_t readVal;
119
120     SS1_SetValue(false);
121     SPI1_Exchange8bit(0x80 | address); //lecture (0=> premier bit à '1')
122     SPI1_Exchange8bit(0x00);
123     readVal = SPI1_Exchange8bit(DUMMY);
124     SS1_SetValue(true);
125
126     return (readVal);
127 }
128 // write register
129 void writeRegister8 (uint8_t address, uint8_t data)
130 {
131     SS1_SetValue(false);
132     SPI1_Exchange8bit(address);
133     SPI1_Exchange8bit(data);
134     SS1_SetValue(true);
135 }
136 // delay Xms
137 void delay(uint8_t Xms)
138 {
139     uint32_t i;
140     while (Xms--)
141     {
142         i = 8000;
143         while (i--);
144     }
145 }
146

```

```

147 // Set the interrupts polarity
148 void MC3419_Pol_INT(MC3419_GPIO_CTRL GPIO_CTRL)
149 {
150     uint8_t value;
151     MC3419_SetMode(STANDBY);
152     value = ReadRegister8(addr_GPIO_CTRL);
153     value &= Mask_GPIO_CTRL;
154     value |= GPIO_CTRL;
155     writeRegister8(addr_GPIO_CTRL, value);
156 }
157
158 // Set interrupt enable
159 void MC3419_INT_Enable (MC3419_INTR_CTRL INTR_CTRL)
160 {
161     uint8_t value;
162     MC3419_SetMode(STANDBY);
163     value = ReadRegister8(addr_INTR_CTRL);
164     value &= Mask_INT_CTRL;
165     value |= INTR_CTRL;
166     writeRegister8(addr_INTR_CTRL, value);
167 }
168
169 // Set motion control
170 void MC3419_MotionCTRL (MC3419_MotionControl Motion)
171 {
172     MC3419_SetMode(STANDBY);
173     writeRegister8(addr_MOTION_CTRL, Motion);
174 }
175
176 // Status Register
177 // lecture registre bits de statuts
178 uint8_t MC3419_ReadStatusRegister (void)
179 {
180     return ReadRegister8(addr_INTR_STAT);
181 }
182
183 // Interrupt pending
184 // lit le interrupt status register et quittance tous les bits
185 uint8_t MC3419_ReadIntStatusRegisterAndAck (void)
186 {
187     uint8_t valueRead;
188
189     valueRead = ReadRegister8(addr_INTR_STAT);
190     writeRegister8(addr_INTR_STAT, 0);
191     return valueRead;
192 }
193
194 // Set FIFO control
195 uint8_t MC3419_FiFo_CTRL_REG (MC3419_FIFO_CTRL ValCTRL)
196 {
197     uint8_t value;
198     uint8_t valueRead;
199
200     valueRead = ReadRegister8(addr_FIFO_CTRL);
201     value = valueRead;
202     value |= ValCTRL;
203     writeRegister8(addr_FIFO_CTRL, value);
204     return valueRead;
205 }
206
207 void MC3419_FIFO_TRESHOLD (uint8_t Fifo_Tresh)
208 {
209     uint8_t value;
210     uint8_t valueRead;
211
212     valueRead = ReadRegister8(addr_FIFO_CTRL_TR);
213     value = valueRead;
214     value &= Mask_FIFO_TH;
215     value |= Fifo_Tresh;
216     writeRegister8(addr_FIFO_CTRL_TR, value);
217 }
218
219 void MC3419_WriteShakeThresholdRegister (uint16_t Value)

```

```

220 {
221     uint8_t Value_8Bit;
222
223     Value_8Bit = Value & 0x00FF;
224     writeRegister8(addr_SHK_LSB, Value_8Bit);
225     Value_8Bit = Value >> 8;
226     writeRegister8(addr_SHK_MSB, Value_8Bit);
227 }
228
229 void MC3419_WriteShakeDurationAndP2PRegister (uint8_t shakeDuration, uint16_t
shakeP2P)
230 {
231     uint16_t value16Bits;
232     uint8_t Value_8Bits;
233
234     //concaténation en une valeur 16 bits
235     value16Bits = (((uint16_t)shakeDuration)<<12 | shakeP2P ) & 0x7FFF;
236
237     //écriture LSBs
238     Value_8Bits = value16Bits & 0x00FF;
239     writeRegister8(addr_PK_P2P_DUR_LSB, Value_8Bits);
240
241     //écriture MSBs
242     Value_8Bits = value16Bits >> 8;
243     writeRegister8(addr_PK_P2P_DUR_MSB, Value_8Bits);
244 }
245

```